

PROJECT REPORT ON
SECURING DATA WITH BLOCK CHAIN AND AI

Submitted by

GONTUMUKKALA RISHITHA

(Regd.No:Y22MC20018)

In partial fulfillment of the requirements for the requirement for the award of the degree of

MASTER of COMPUTER APPLICATIONS

Under the Esteemed Guidance of

Dr. G. NEELIMA

Assistant Professor

Department of Computer Science and Engineering

Acharya Nagarjuna University



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

ACHARYA NAGARJUNA UNIVERSITY

Nagarjuna Nagar, Guntur,

Andhra Pradesh, India.

June 2023

ACHARYA NAGARJUNA UNIVERSITY
NAGARJUNA NAGAR, GUNTUR.
Department of Computer Science & Engineering.



BONAFIDE CERTIFICATE

This is to certify that the Project Report entitled “**SECURING DATA WITH BLOCK CHAIN AND AI**” is a bonafide record of the project work submitted by **Gontumukkala Rishitha (Y22MC20018)** during the year (2021-2023) in partial fulfillment of the requirements for the award of the degree **Master of Computer Applications** in the **Department of Computer Science & Engineering**.

Acharya Nagarjuna University

Head of the department

Project Guide

External Examiner

DECLARATION

I **GONTUMUKKALA RISHITHA**, bearing Regd.no: **Y22MC20018** a student of DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING, UNIVERSITY COLLEGE OF SCIENCES, ACHARAYA NAGARJUNA UNIVERSITY, in partial fulfillment of the requirement for the award of the degree of Master Of Computer Applications (**MCA**) declare that I own the full responsibility for the formation and results provided in this report titled “**SECURING DATA WITH BLOCKCHAIN AND AI**” under the supervision of **Dr.G.Neelima**, Assistant Professor, Department of Computer Science & Engineering, Acharya Nagarjuna University.

Name: GONTUMUKKALA RISHITHA

Regd.no: Y22MC20018

Date: 30/05/2023

PROJECT COMPLETION CERTIFICATE

This is to certify that **GONTUMUKKALA RISHITHA (Y22MC20018)**, MCA(Master of Computer Applications) student of **ACHARYA NAGARJUNA UNIVERSITY**, have successfully completed her project work title coined "**Securing Data with Block Chain and AI**" as part of her course curriculum.

She accomplished her project using Machine Learning with Python Technologies during the period **February 2023 - May 2023** under the guidance and supervision of **V.Upendra** (project Guide) for **VXL IT SOLUTIONS**.

She has completed the assigned Project well within the time frame.

She is sincere, hardworking and her conduct during period is commendable.

We wish the very best for her career and future endeavors.


Authorized Signatory



ACKNOWLEDGEMENT

It is a great pleasure to have the opportunity to extend our heartfelt gratitude to everyone who helped us throughout the course of this project. I profoundly grateful to our supervisor **Dr.G.Neelima** for all the support and encouragement she gave me. Her friendly guidance and expert advice have been invaluable throughout all stages of the work. Without her guidance and constant feedback these Project works not have been achievable.

I would also wish to express my gratitude to **Prof.K.Gangadhara Rao** for extended discussions and valuable suggestions which have contributed greatly to the improvement of the thesis.

I must also thank my parents and friends for the immense support and help during this project. Without their help, completing this project would have been very difficult.

Name: Gontumukkala Rishitha

Regd.no: Y22MC20018

ABSTRACT

Data is the input for various artificial intelligence (AI) algorithms to mine valuable features, yet data in Internet is scattered everywhere and controlled by different stakeholders who cannot believe in each other, and usage of the data in complex cyberspace is difficult to authorize or to validate. As a result, it is very difficult to enable data sharing in cyberspace for the real big data, as well as a real powerful AI. In this paper, we propose the SecNet, an architecture that can enable secure data storing, computing, and sharing in the large-scale Internet environment, aiming at a more secure cyberspace with real big data and thus enhanced AI with plenty of data source, by integrating three key components:

- 1) Blockchain-based data sharing with ownership guarantee, which enables trusted data sharing in the large-scale environment to form real big data;
- 2) AI-based secure computing platform to produce more intelligent security rules, which helps to construct a more trusted cyberspace;
- 3) Trusted value-exchange mechanism for purchasing security service, providing a way for participants to gain economic rewards when giving out their data or service, which promotes the data sharing and thus achieves better performance of AI. Moreover, we discuss the typical use scenario of SecNet as well as its potentially alternative way to deploy, as well as analyze its effectiveness from the aspect of network security and economic revenue.

List of Figures

S. No.	Fig. No.	Fig. Name	Page No.
1	2.6.1	Django Life Cycle.	19
2	2.6.2	MVT architecture	20
3	3.3.1	ER Diagram	27
4	3.4.1.1	Use case Diagram-Patient	29
5	3.4.1.2	Use case Diagram-Hospital	30
6	3.4.2	Sequence Diagram	31
7	3.4.3	Class Diagram	32
8	6.1	Index page	65
9	6.2	Patient New Registration	66
10	6.3	Patient Profile Creation	67
11	6.4	Hospital1 Login	68
12	6.5	Hospital1 Welcome page	69
13	6.6	Access Patient share Data	70
14	6.7	Patient Data Access	71
15	6.8	Hospital2 Login	72
16	6.9	Hospital1 Welcome page	73
17	6.10	Access Patient share Data	74
18	6.11	Access Data	75
18	6.12	Patient Login	76
19	6.13	Patient ID1 data	77

List of Tables

S. No.	Table No.	Table Name	Page No.
1	3.2.1	Patients	25

INDEX

Content	Page No
Chapter 1: Introduction	1
1.1 Blockchain	2
1.2 Artificial Intelligence	3
1.3 Rewards	3
Chapter 2: System Analysis	4
2.1 Existing System	5
2.2 Proposed System	5
2.3 FEASIBILITY STUDY	5
2.3.1 ECONOMICAL FEASIBILITY	6
2.3.2 TECHNICAL FEASIBILITY	6
2.3.3 SOCIAL FEASIBILITY	6
2.4 Hardware Requirements	7
2.5 Software Requirements	7
2.6 Justification of Platform	8
Chapter 3: System Design	13
3.1 Module Division	14
3.2 Data Dictionary	15
3.3 E-R diagrams	15
3.4 UML Diagrams	17
3.4.1 Use case Diagram	18
3.4.2 Sequence Diagram	20
3.4.3 Class Diagram	22
Chapter 4: Implementation	23

4.1 Modules	24
4.2 Code	26
Chapter 5: Testing	49
5.1 Testing Approach	50
5.1.1 Unit testing	50
5.1.2 Integration testing	50
5.1.3 Functional test	51
Chapter 6: Results and Discussions	54
Chapter 7: Conclusion	68
Chapter 8: References	70

Chapter 1

Introduction

1. Introduction

In cyber world everything is dependent on data and all Artificial Intelligence algorithms discover knowledge from past data only, for example in online shopping application users review data is very important for new comers to take decision on which product to purchase or not to purchase, we can take many examples like health care to know good hospitals or education institutions etc. Not all cyber data can be made publicly available such as Patient Health Data which contains patient disease details and contact information and if such data available publicly then there is no security for that patient data.

Now a days all service providers such as online social networks or cloud storage will store some type of user's data and they can sale that data to other organization for their own benefits and user has no control on his data as that data is saved on third party servers.

To overcome from above issue author has describe concept called Private Data Centers (PDC) with Blockchain and AI technique to provide security to user's data. In this technique 3 functions will work which describe below

1.1 Blockchain: Blockchain-based data sharing with ownership guarantee, which enables trusted data sharing in the large-scale environment to form real big data. In this technique users can define access control which means which user has permission to access data and which user cannot access data and Blockchain object will be generate on that access data and allow only those users to access data which has permissions. In Blockchain object user will add/subscribe share data and give permission.

1.2 Artificial Intelligence: AI-based secure computing platform to produce more intelligent security rules, which helps to construct more trusted cyberspace. AI work similar to human brain and responsible to execute logic to check whether requesting user has permission to access shared data. If access is available then AI allow Blockchain to display share data otherwise ignore request.

1.3 Rewards: In this technique all users who are sharing the data will earn rewards point upon any user access his data trusted value-exchange mechanism for purchasing security service, providing way for participants to gain economic rewards when giving out their data or service, which promotes the data sharing and thus achieves better performance of AI.

To implement this project author has taken medical data sharing example and I am also using same concept to build this project.

Chapter 2

System Analysis

2.1 Existing System

- In existing system data protection mechanisms such as encryption was failed in securing the data from the attacker.
- It does not verify whether the user was authorized or not.

2.2 Proposed System

- The proposed system enables Private Data Centers (PDC) with Blockchain and AI technique to provide security to user's data.
- Blockchain-based data sharing with ownership guarantee, which enables trusted data sharing and generates hash code (unique).

2.3 FEASIBILITY STUDY

The feasibility of the project is analyzed in this phase and business proposal is put forth with a very general plan for the project and some cost estimates. During system analysis the feasibility study of the proposed system is to be carried out. This is to ensure that the proposed system is not a burden to the company. For feasibility analysis, some understanding of the major requirements for the system is essential.

Three key considerations involved in the feasibility analysis are,

◆ **ECONOMICAL FEASIBILITY**

◆ **TECHNICAL FEASIBILITY**

◆ **SOCIAL FEASIBILITY**

2.3.1 ECONOMICAL FEASIBILITY

This study is carried out to check the economic impact that the system will have on the organization. The amount of fund that the company can pour into the research and development of the system is limited. The expenditures must be justified. Thus, the developed system as well within the budget and this was achieved because most of the technologies used are freely available. Only the customized products had to be purchased.

2.3.2 TECHNICAL FEASIBILITY

This study is carried out to check the technical feasibility, that is, the technical requirements of the system. Any system developed must not have a high demand on the available technical resources. This will lead to high demands on the available technical resources. This will lead to high demands being placed on the client. The developed system must have a modest requirement, as only minimal or null changes are required for implementing this system.

2.3.3 SOCIAL FEASIBILITY

The aspect of study is to check the level of acceptance of the system by the user. This includes the process of training the user to use the system efficiently. The user must not feel threatened by the system, instead must accept it as a necessity. The level of acceptance by the users solely depends on the methods that are employed to educate the user about the system and to make him familiar with it. His level of confidence must be raised so that he is also able to make some constructive criticism, which is welcomed, as he is the final user of the system.

2.4 Hardware Requirements

Developing Time Requirements

- Intel ® Core i3 CPU 3.0 GHz or above
- 4GMRAM
- 500GB Hard Disk Drive (HDD) + 24GB Solid State Drive(SSD)
- Monitor

Runtime Requirements

- **Processor:** Intel Atom® Processor Z2520 1.2 GHz, or faster processor
- **Storage:** Between 850 MB and 1.2 GB, depending on the language version
- **RAM:** Minimum of 512 MB, 2 GB is recommended
- **Hard Disk:** 2GB of available hard-disk space for installation; extra free space is required during installation.

2.5 Software Requirements

- **IDE:** PyCharm
- **Front End:** Html, CSS, JS
- **Back End:** MySQL Server
- **Operating System:** Window 7/Window 10
- **Programming Language:** Python

2.6 Justification of Platform

PYTHON

Python is a general-purpose interpreted, interactive, object-oriented, and high-level programming language. An interpreted language, Python has a design philosophy that emphasizes code readability (notably using whitespace indentation to delimit code blocks rather than curly brackets or keywords), and a syntax that allows programmers to express concepts in fewer lines of code than might be used in languages such as C++ or Java. It provides constructs that enable clear programming on both small and large scales. Python interpreters are available for many operating systems. CPython, the reference implementation of Python, is open-source software and has a community-based development model, as do nearly all its variant implementations. CPython is managed by the non-profit Python Software Foundation. Python features a dynamic type of system and automatic memory management. It supports multiple programming paradigms, including object-oriented, imperative, functional and procedural, and has a large and comprehensive standard library.

Python is a multi-paradigm programming language. Object-oriented programming and structured programming are fully supported, and many of its features support functional programming and aspect-oriented programming (including by Meta programming and meta-objects (magic methods)). Many other paradigms are supported via extensions, including design by contract and logic programming.

Python uses dynamic typing, and a combination of reference counting and a cycle-detecting garbage collector for memory management. It also features dynamic name resolution (late binding), which binds method and variable names during program execution.

DJANGO

Django is a high-level Python Web framework that encourages rapid development and clean, pragmatic design. Built by experienced developers, it takes care of much of the hassle of Web development, so you can focus on writing your app without needing to reinvent the wheel. It is free and open source.

Django's primary goal is to ease the creation of complex, database-driven websites. Django emphasizes reusability and "pluggability" of components, rapid development, and the principle of do not repeat yourself.

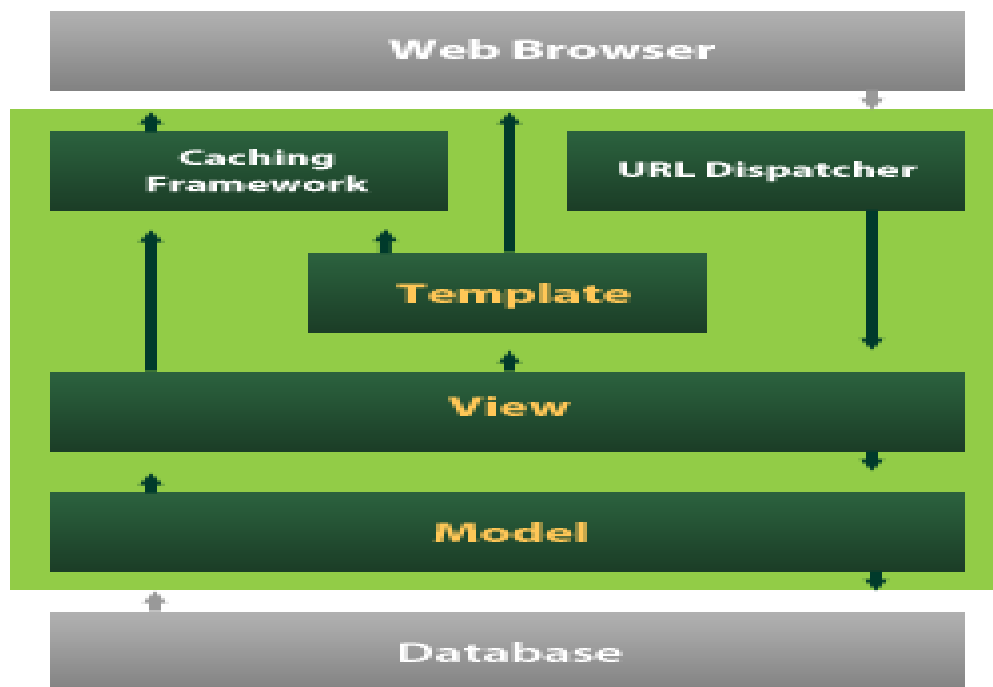


Fig.2.6.1 Django Life Cycle.

Django also provides an optional administrative create, read, update and delete interface that is generated dynamically through introspection and configured via admin models.

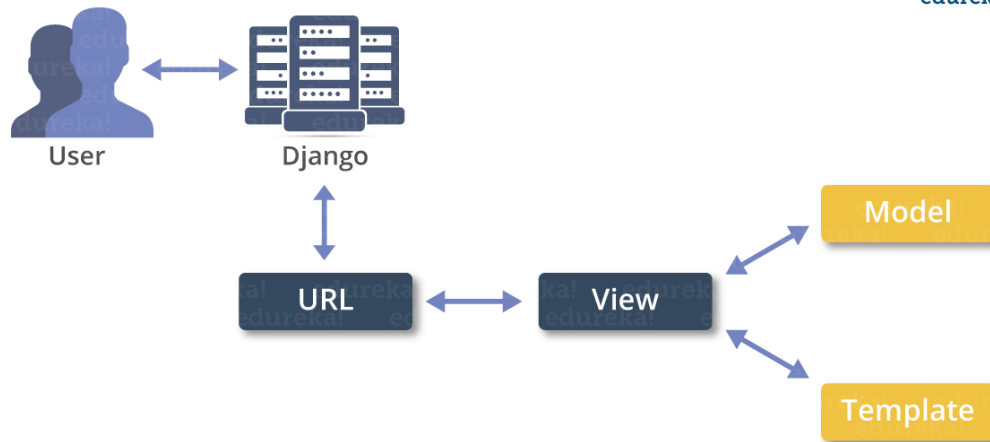


Fig.2.6.2 MVT architecture.

Despite having its own nomenclature, such as naming the callable objects generating the HTTP responses "views", the core Django framework can be seen as MVT architecture. It consists of an object-relational mapper (ORM) that mediates between data models (defined as Python classes) and a relational database ("Model"), a system for processing HTTP requests with a web templating system("View"), and a regular-expression-based URL dispatcher ("Controller").

Also included in the core framework are:

- A lightweight and standalone web server for development and testing
- A form serialization and validation system that can translate between HTML forms and values suitable for storage in the database
- A template system that utilizes the concept of inheritance borrowed from object-oriented programming
- A caching framework that can use any of several cache methods
- Support for middleware classes that can intervene at various stages of request processing

and carry out custom functions

- An internal dispatcher system that allows components of an application to communicate events to each other via pre-defined signals.
- An internationalization system, including translations of Django's own components into a variety of languages
- A system for extending the capabilities of the template engine
- An interface to Python's built-in unit test framework

MySQL Database

MySQL is a fast, easy-to-use RDBMS being used for many small and big businesses. MySQL is developed, marketed and supported by MySQL AB, which is a Swedish company. MySQL is becoming so popular because of many good reasons,

- MySQL is released under an open-source license. So, user has nothing to pay to use it.
- MySQL is a very powerful program in its own right. It handles a large subset of the functionality of the most expensive and powerful database packages.
- MySQL uses a standard form of the well-known SQL data language.
- MySQL works on many operating systems and with many languages including PHP, PERL, C, C++, JAVA, Python etc.
- MySQL works very quickly and works well even with large datasets.
- MySQL is very friendly to PHP, the most appreciated language for web development.

MySQL supports large databases, up to 50 million rows or more in a table. The default file size limit

for a table is 4GB, but user can increase this (if user's operating system can handle it) to a theoretical limit of 8 million terabytes (TB).

Chapter 3

System Design

3.1 Module Division

Description of the Modules

- **Patients:** Patients first create his profile with all disease details and then select desired hospital with whom he wishes to share/subscribe data. While creating profile application will create Blockchain object with allowable permission and it will allow only those hospitals to access data.
- **Patient Login:** Patient can login to application with his profile id and check total rewards he earned from sharing data.
- **Hospital:** Hospital1 and Hospital2 are using in this application as two organizations with whom patient can share data. At a time, any hospital can login to application and then enter search string as disease name.

AI algorithm will take input disease string and then perform search operation on all patients to get similar disease patients and then check whether this hospital has permission to access that patient data or not, if hospital has access permission, then it will display those patients records to that hospital.

3.2 Data Dictionary

PATIENTS:

Column	Type	Null
patient_id	Int	No
patient_name	varchar (50)	No
Age	Int	No
problem_desc	varchar (500)	No
profile_date	Date	No
access_data	varchar (200)	No
Gender	varchar (30)	No
contact_no	varchar (12)	No
Address	varchar (100)	No
blockchain_hash	varchar (200)	No
Revenue	double	No

Fig.3.2.1

3.3 E-R diagrams

- ◆ A database is a collection of entities or a relation among entities.
- ◆ **Entity:** An entity is an individual object that exists and distinguishes from other individuals.
- ◆ Entities have attributes
 - Examples: people have names and addresses.

- ◆ **Entity Set:** An entity set is a set of entities of the same type that share the same properties.
- ◆ **Attributes:** An entity is represented by the set of attributes, i.e. descriptive properties possessed by all members of entity set.
- **Types of attributes:**
 - Single and Composite attributes
 - Single-valued and multi-valued attributes
 - Derived attributes
- **Relationship:** A relationship is an association among several entities.
- **Relationship Set:** A relationship set is a mathematical relation among $n \geq 2$ entities, each taken from entity sets.
- **Weak Entity:** A weak entity is an entity that must be defined by a foreign key relationship with another entity as it cannot be uniquely identified by its own attributes alone.
- **Primary Key:** A minimal super key for each entity set among its super keys; the minimal super key is termed as the entity set's primary key.
- **Super Key:** A super key is a set of one or more attributes that, taken collectively, allow us to identify uniquely an entity in the entity set.

E-R diagram representation:

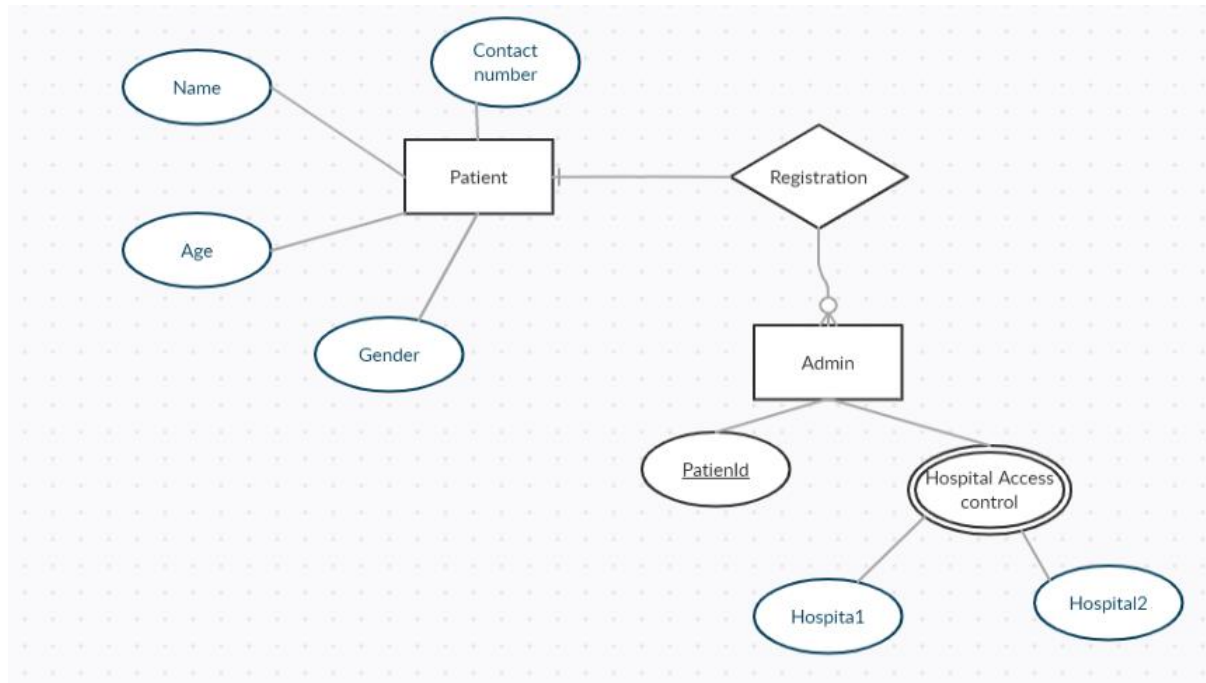


Fig.3.3.1 ER diagram

3.4 UML Diagrams

The Unified Modeling Language (UML) is a standardized visual specification language for object modeling. UML is a general-purpose modeling language that includes a graphical notation used to create an abstract model of a system, referred to as a UML model. The goal is for UML to become a common language for creating models of object-oriented computer software. In its current form UML is comprised of two major components: a Meta- model and a notation. In the future, some form of method or process may also be added to; or associated with, UML. The Unified Modeling Language is a standard language for specifying, Visualization, Constructing and documenting the artifacts of the software system as well as for business modeling and other non-software systems. The UML

represents a collection of best engineering practices that proven successful in the modeling of large and complex systems. The UML is a very important part of developing objects-oriented software and the software development process. The UML uses mostly graphical notations to express the design of software projects.

Goals:

The primary goals in the design of the UML are as follows:

- Provides users a ready-to-use, expensive visual modeling Language so that they can develop and exchange meaningful models.
- Provide extendibility and specialization mechanisms to extend the core concepts.
- Be independent of particular programming languages and development process.
- Provide a formal basis for understanding the modeling language.
- Encourage the growth of OO tools methods.
- Support higher level development concepts such as collaborations, frameworks, patterns and components.
- Integrate best practices.

3.4.1 Use case Diagram

A use case diagram is the Unified Modeling Language (UML) is a type of behavioral diagram defined is and created from a Use-case analysis. A use case diagram shows a set of use cases and actors and their relationships. Users apply the use case diagram to illustrate the static and use case view of a system. Use case diagram are especially important in organizing and modeling the behaviors of a

system.

Use case For Patient

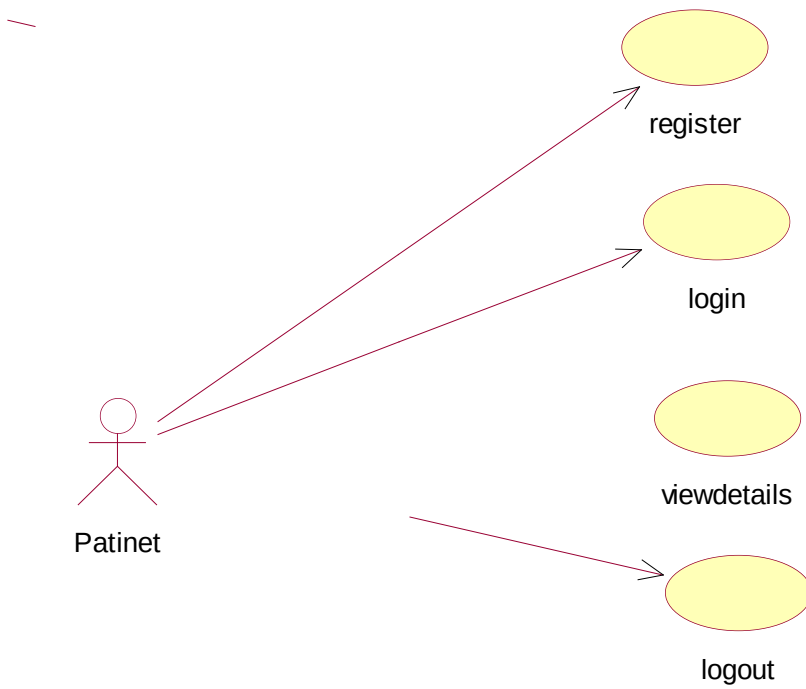


Fig.3.4.1.1 Use case for patient

Use case for Hospital

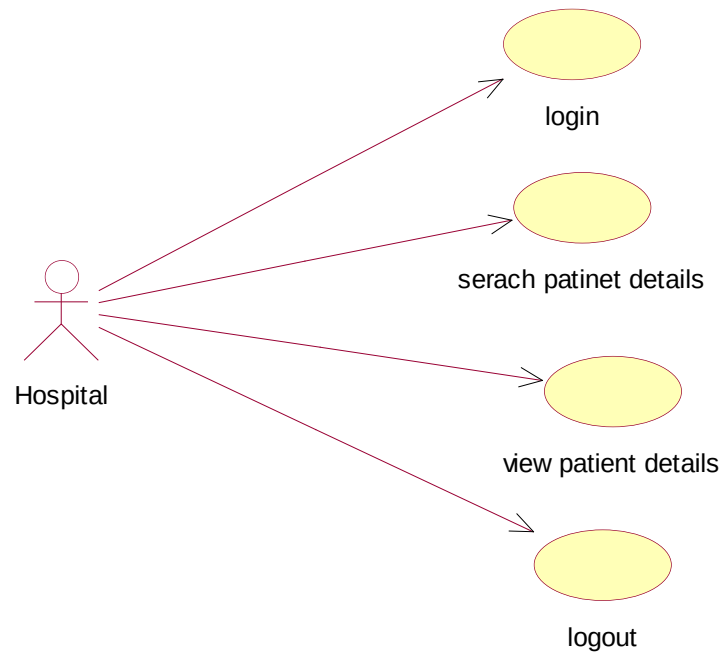


Fig.3.4.1.2 Use case for Hospital.

3.4.2 Sequence Diagram

A sequence diagram in a unified modeling language is kind of interaction that shows how processes operate with one another in what order. It is a construct of message sequence chart. Sequence diagrams are sometimes called as event diagrams or event scenarios, and timing diagrams.

Sequence Diagram

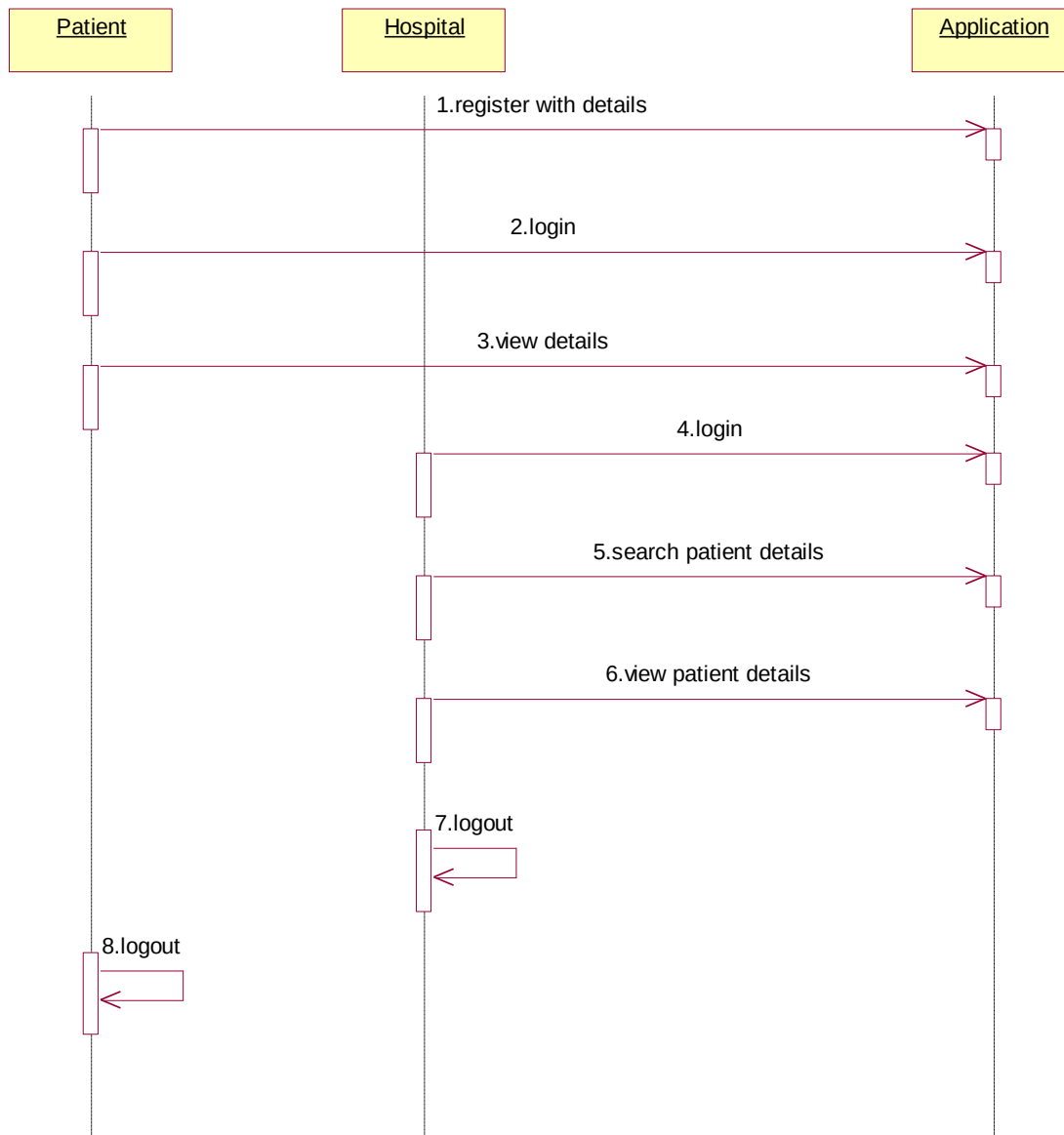


Fig.3.4.2 Sequence Diagram

3.4.3 Class Diagram

Class diagram is a static diagram. It represents the static view of an application. Class diagram is not only used for visualizing, describing, and documenting different aspects of a system but also for constructing executable code of the software application.

Class diagram describes the attributes and operations of a class and also the constraints imposed on the system. The class diagrams are widely used in the modeling of object-oriented systems because they are the only UML diagrams, which can be mapped directly with object-oriented languages.

Class diagram shows a collection of classes, interfaces, associations, collaborations, and constraints. It is also known as a structural diagram.

Class Diagram



Fig.3.4.3 Class Diagram

Chapter 4

Implementation

4.1 Modules

To implement this project author has taken medical data sharing example and I am doing same concept to build this project.

Modules Information:

This project consists of two modules

1) Patients:

Patients first create his profile with all disease details and then select desired hospital with whom he wishes to share or subscribe data. While creating profile application will create blockchain object with allowable permission and it will allow only those hospitals to access the data.

Patient Login:

Patient can login to application with his profile id and check total rewards he earned from sharing data.

2) Hospital:

Hospital 1 and Hospital 2 are using in this application as two organizations with whom patient can share data. At a time any hospital can login to application and then enter search string as disease name.

AI algorithm will take input disease string and then perform search operation on all the patients to get similar disease patients and then check whether this hospital has permission to access that patient data or not, if hospital has access permission then it will display those patients records to that hospital.

Below is the code example to create blockchain object with patient data.

```
blockchain = Blockchain()
```

4.2 Code

Index.html:

```
{% load static %}
<html>
<head>
<title>Securing Data With Blockchain and AI</title>
<meta http-equiv="content-type" content="text/html; charset=utf-8" />
<link href="{% static 'style.css' %}" rel="stylesheet" type="text/css" />
</head>
<body>
<div class="main">
<div class="main_resize">
<div class="header">
<div class="logo">
<h1><span>Securing Data With Blockchain and AI</span><small></small></h1>
</div>
</div>
<div class="content">
<div class="content_bg">
<div class="menu_nav">
<ul>
<ul>
<li><a href="{% url 'index' %}">Home</a></li>
<li><a href="{% url 'Hospital' %}">Hospital</a></li>
<li><a href="{% url 'Patient' %}">Patients Login</a></li>
<li><a href="{% url 'CreateProfile' %}">New Patient Register Here</a></li>
</ul>
</ul>
</div>
<div class="hbg"><imgsrc="{% static 'images/header_images.jpg' %}" width="915"
height="286" alt="" /></div>

{{ data }}

<p align="justify"><font size="3" style="font-family: Comic Sans MS"
color="black">Abstract-Data is the input for various articial intelligence (AI)
algorithms to mine valuable features, yet
data in Internet is scattered everywhere and controlled by different stakeholders
who cannot believe in each
other, and usage of the data in complex cyberspace is difcult to authorize or to
validate. As a result, it is very
difcult to enable data sharing in cyberspace for the real big data, as well as a
real powerful AI. </p>
<p align="justify"><font size="3" style="font-family: Comic Sans MS" color="black">
In this paper,
we propose the SecNet, an architecture that can enable secure data storing,
computing, and sharing in the
large-scale Internet environment, aiming at a more secure cyberspace with real big
data and thus enhanced
AI with plenty of data source, by integrating three key components:</p>
<p align="justify"><font size="3" style="font-family: Comic Sans MS" color="black">
1) blockchain-based data sharing with
ownership guarantee, which enables trusted data sharing in the large-scale
environment to form real big data. <br/>
```

2) AI-based secure computing platform to produce more intelligent security rules, which helps to construct a more trusted cyberspace.

3) trusted value-exchange mechanism for purchasing security service, providing a way for participants to gain economic rewards when giving out their data or service, which promotes the data sharing and thus achieves better performance of AI. Moreover, we discuss the typical use scenario of SecNet as well as its potentially alternative way to deploy, as well as analyze its effectiveness from the aspect of network security and economic revenue. </p>

</body>
</html>

Hospital.html:

```
{% load static %}
<html>
<head>
<title>Securing Data With Blockchain and AI</title>
<meta http-equiv="content-type" content="text/html; charset=utf-8" />
<link href="{% static 'style.css' %}" rel="stylesheet" type="text/css" />
<script language="javascript">
    function validate(formObj)
    {
        if(formObj.t1.value.length==0)
        {
            alert("Please Enter Username");
            formObj.t1.focus();
            return false;
        }
        if(formObj.t2.value.length==0)
        {
            alert("Please Enter Password");
            formObj.t2.focus();
            return false;
        }
        formObj.actionUpdateData.value="update";
        return true;
    }
</script>
<script language="javascript" type="text/javascript" src="datetimepicker.js">
</script>
</head>
<body>
<div class="main">
<div class="main_resize">
<div class="header">
<div class="logo">
<h1><span>Securing Data With Blockchain and AI</span><small></small></h1>
</div>
</div>
<div class="content">
<div class="content_bg">
<div class="menu_nav">
<ul>
<li><a href="{% url 'index' %}">Home</a></li>
<li><a href="{% url 'Hospital' %}">Hospital</a></li>
<li><a href="{% url 'Patient' %}">Patients Login</a></li>
<li><a href="{% url 'CreateProfile' %}">New Patient Register Here</a></li>
</ul>
</div>
<div class="hbg"><imgsrc="{% static 'images/header_images.jpg' %}" width="915"
height="286" alt="" /></div>
<center>
<form name="f1" method="post" action="{% url 'HospitalLogin' %}" onsubmit="return
validate(this);">
{% csrf_token%}<br/>
<h2><b>Hospital Login Screen</b></h2>

<font size="" color="red"><center>{{ data }}</center></font>
```

```

        <table align="center" width="30%" >
        <tr><td><b>Username</b></td>
        <td><input type="text" name="t1" style="font-family: Comic Sans
MS" size="30"></td></tr>

        <tr><td><b>Password</b></td><td><input name="t2" type="password"
size="30"></td></tr>

        <tr><td></td><td><input type="submit" value="Login"></td>
        </table>
        </div>

        </div>

    </body>
</html>

```

HospitalScreen.html:

```
{% load static %}
<html>
<head>
<title>Securing Data With Blockchain and AI</title>
<meta http-equiv="content-type" content="text/html; charset=utf-8" />
<link href="{% static 'style.css' %}" rel="stylesheet" type="text/css" />
</head>
<body>
<div class="main">
<div class="main_resize">
<div class="header">
<div class="logo">
<h1><span>Securing Data With Blockchain and AI</span><small></small></h1>
</div>
</div>
<div class="content">
<div class="content_bg">
<div class="menu_nav">
<ul>
<ul>
<li><a href="{% url 'AccessData' %}">Access Patient Share Data</a></li>
<li><a href="{% url 'index' %}">Logout</a></li>
</ul>
</ul>
</div>
<div class="hbg"><imgsrc="{% static 'images/header_images.jpg' %}" width="915"
height="286" alt="" /></div>

<font size="3" color="red"><center>{{ data }}</center></font></body>
</html>
```

AccessData.html:

```
{% load static %}
<html>
<head>
<title>Securing Data With Blockchain and AI</title>
<meta http-equiv="content-type" content="text/html; charset=utf-8" />
<link href="{% static 'style.css' %}" rel="stylesheet" type="text/css" />
<script language="javascript">
    function validate(formObj)
    {
        if(formObj.t1.value.length==0)
        {
            alert("Please Enter search data");
            formObj.t1.focus();
            return false;
        }
        formObj.actionUpdateData.value="update";
        return true;
    }
</script>
<script language="javascript" type="text/javascript" src="datetimepicker.js">
</script>
</head>
<body>
<div class="main">
<div class="main_resize">
<div class="header">
<div class="logo">
<h1><span>Securing Data With Blockchain and AI</span><small></small></h1>
</div>
</div>
<div class="content">
<div class="content_bg">
<div class="menu_nav">
<ul>
<li><a href="{% url 'AccessData' %}">Access Patient Share Data</a></li>
<li><a href="{% url 'index' %}">Logout</a></li>
</ul>
</div>
<div class="hbg"><imgsrc="{% static 'images/header_images.jpg' %}" width="915"
height="286" alt="" /></div>
<center>
<form name="f1" method="post" action="{% url 'PatientDataAccess' %}" onsubmit="return
validate(this);">
{% csrf_token%}<br/>
<h2><b>Patient Share Data Access Screen</b></h2>

<font size="" color="red"><center>{{ data }}</center></font>

<table align="center" width="30%" >
<tr><td><b>AI&nbsp;Search&nbsp;String</b></td>
<td><input type="text" name="t1" style="font-family: Comic Sans
MS" size="30"></td></tr>

<tr><td></td><td><input type="submit" value="Access Data"></td>
</tr>
</table>
```



```

    </div>
  </div>
</body>
</html>
```

ViewAccessData.html:

```
{% load static %}
<html>
<head>
<title>Securing Data With Blockchain and AI</title>
<meta http-equiv="content-type" content="text/html; charset=utf-8" />
<link href="{% static 'style.css' %}" rel="stylesheet" type="text/css" />
<script language="javascript">
    function validate(formObj)
    {
        if(formObj.t1.value.length==0)
        {
            alert("Please Enter search data");
            formObj.t1.focus();
            return false;
        }
        formObj.actionUpdateData.value="update";
        return true;
    }
</script>
<script language="javascript" type="text/javascript" src="datetimepicker.js">
</script>
</head>
<body>
<div class="main">
<div class="main_resize">
<div class="header">
<div class="logo">
<h1><span>Securing Data With Blockchain and AI</span><small></small></h1>
</div>
</div>
<div class="content">
<div class="content_bg">
<div class="menu_nav">
<ul>
<li><a href="{% url 'AccessData' %}">Access Patient Share Data</a></li>
<li><a href="{% url 'index' %}">Logout</a></li>
</ul>
</div>

    <div class="hbg"><imgsrc="{% static 'images/header_images.jpg' %}"
width="915" height="286" alt="" /></div>
        <center>

        <br/>
        {{ data|safe }}

    </div>
</body>
</html>
```

CreateProfile.html:

```
{% load static %}
<html>
<head>
<title>Securing Data With Blockchain and AI</title>
<meta http-equiv="content-type" content="text/html; charset=utf-8" />
<link href="{% static 'style.css' %}" rel="stylesheet" type="text/css" />
<script language="javascript">
    function validate(formObj)
    {
        if(formObj.t1.value.length==0)
        {
            alert("Please Enter patient name");
            formObj.t1.focus();
            return false;
        }
        if(formObj.t2.value.length==0)
        {
            alert("Please Enter Age");
            formObj.t2.focus();
            return false;
        }
        if(isNaN(formObj.t2.value)){
            alert("Age must be numeric");
            formObj.t2.focus();
            return false;
        }
        if(formObj.t3.value.length==0)
        {
            alert("Please Enter Problem Description");
            formObj.t3.focus();
            return false;
        }

        if(formObj.t6.value.length==0)
        {
            alert("Please Enter Contact No");
            formObj.t6.focus();
            return false;
        }
        if(isNaN(formObj.t6.value)){
            alert("Contact No must be numeric");
            formObj.t6.focus();
            return false;
        }
        if(formObj.t7.value.length==0)
        {
            alert("Please Enter Address");
            formObj.t7.focus();
            return false;
        }
        formObj.actionUpdateData.value="update";
        return true;
    }
</script>
```

```

        <script language="javascript" type="text/javascript" src="datetimepicker.js">
</script>
</head>
<body>
<div class="main">
<div class="main_resize">
<div class="header">
<div class="logo">
<h1><span>Securing Data With Blockchain and AI</span><small></small></h1>
</div>
</div>
<div class="content">
<div class="content_bg">
<div class="menu_nav">
<ul>
<li><a href="{% url 'index' %}">Home</a></li>
<li><a href="{% url 'Hospital' %}">Hospital</a></li>
<li><a href="{% url 'Patient' %}">Patients Login</a></li>
<li><a href="{% url 'CreateProfile' %}">New Patient Register Here</a></li>
</ul>
</div>
<div class="hbg"><imgsrc="{% static 'images/header_images.jpg' %}" width="915"
height="286" alt="" /></div>
<center>
<form name="f1" method="post" action="{% url 'CreateProfileData' %}" onsubmit="return
validate(this);">
{% csrf_token%}<br/>
<h2><b>Patients Profile Creation Screen</b></h2>

        <font size="" color="red"><center>{{ data }}</center></font>

                <table align="center" width="30%" >
                    <tr><td><b>Patient&nbsp;Name</b></td>
                    <td><input type="text" name="t1" style="font-family: Comic Sans
MS" size="30"></td></tr>

                    <tr><td><b>Age</b></td><td><input name="t2" type="Text"
size="10"></td></tr>

                    <tr><td><b>Problem&nbsp;Desc</b></td>
                    <td><textarea name="t3" rows="10" cols="60"></textarea></td></tr>

                    <tr><td><b>Access Control</b></td><td><select name="t4" multiple>
<option value="Hospital1">Hospital 1</option>
<option value="Hospital2">Hospital 2</option>
</select>
</td></tr>

                    <tr><td><b>Gender</b></td><td><select name="t5" style="font-family:
Comic Sans MS">
<option value="Male">Male</option>
<option value="Female">Female</option>
</select>
</td></tr>

                    <tr><td><b>Contact&nbsp;No</b></td><td><input type="text" name="t6"
style="font-family: Comic Sans MS" size=20/></td></tr>

```

```

<tr><td><b>address</b></td><td><textarea name="t7" style="font-family: Comic Sans
MS" rows="6" cols="30"></textarea></td></tr>
      <tr><td></td><td><input type="submit" value="Create"></td>
    </table>
  </div>
</div>
</body>
</html>

```

Patient.html:

```
{% load static %}
<html>
<head>
<title>Securing Data With Blockchain and AI</title>
<meta http-equiv="content-type" content="text/html; charset=utf-8" />
<link href="{% static 'style.css' %}" rel="stylesheet" type="text/css" />
<script language="javascript">
    function validate(formObj)
    {
        if(formObj.t1.value.length==0)
        {
            alert("Please Enter Patient ID");
            formObj.t1.focus();
            return false;
        }

        formObj.actionUpdateData.value="update";
        return true;
    }
</script>
<script language="javascript" type="text/javascript" src="datetimepicker.js">
</script>
</head>
<body>
<div class="main">
<div class="main_resize">
<div class="header">
<div class="logo">
<h1><span>Securing Data With Blockchain and AI</span><small></small></h1>
</div>
</div>
<div class="content">
<div class="content_bg">
<div class="menu_nav">
<ul>
<li><a href="{% url 'index' %}">Home</a></li>
<li><a href="{% url 'Hospital' %}">Hospital</a></li>
<li><a href="{% url 'Patient' %}">Patients Login</a></li>
<li><a href="{% url 'CreateProfile' %}">New Patient Register Here</a></li>
</ul>
</div>
<div class="hbg"><imgsrc="{% static 'images/header_images.jpg' %}" width="915"
height="286" alt="" /></div>
<center>
<form name="f1" method="post" action="{% url 'PatientLogin' %}" onsubmit="return
validate(this);">
{% csrf_token%}<br/>
<h2><b>Patient Login Screen</b></h2>
<font size="" color="red"><center>{{ data }}</center></font>

<table align="center" width="30%" >
<tr><td><b>Patient ID</b></td>
<td><input type="text" name="t1" style="font-family: Comic Sans
MS" size="30"></td></tr>
<tr><td></td><td><input type="submit" value="Login"></td>
```

```
        </table>
        </div>
    </div>
</body>
</html>
```

ViewData.html:

```
{% load static %}
<html>
<head>
<title>Securing Data With Blockchain and AI</title>
<meta http-equiv="content-type" content="text/html; charset=utf-8" />
<link href="{% static 'style.css' %}" rel="stylesheet" type="text/css" />
<script language="javascript">
    function validate(formObj)
    {
        if(formObj.t1.value.length==0)
        {
            alert("Please Enter search data");
            formObj.t1.focus();
            return false;
        }
        formObj.actionUpdateData.value="update";
        return true;
    }
</script>
<script language="javascript" type="text/javascript" src="datetimepicker.js">
</script>
</head>
<body>
<div class="main">
<div class="main_resize">
<div class="header">
<div class="logo">
<h1><span>Securing Data With Blockchain and AI</span><small></small></h1>

</div>
</div>
<div class="content">
<div class="content_bg">
<div class="menu_nav">
<ul>

<li><a href="{% url 'index' %}">Logout</a></li>
</ul>
</div>
<div class="hbg"><imgsrc="{% static 'images/header_images.jpg' %}" width="915"
height="286" alt="" /></div>
                <center>

                <br/>
                {{ data|safe }}

            </body>
</html>
```


Views.py

```
from django.shortcuts import render
from django.template import RequestContext
from django.contrib import messages
import pymysql
from django.http import HttpResponse
import datetime
import json
import time
from hashlib import sha256

class Blockchain:
    # difficulty of our PoW algorithm
    difficulty = 2

    def __init__(self):
self.unconfirmed_transactions = []
self.chain = []
self.create_genesis_block()

    def create_genesis_block(self):
        """
        A function to generate genesis block and appends it to
        the chain. The block has index 0, previous_hash as 0, and
        a valid hash.
        """
genesis_block = Block(0, [], time.time(), "0")
genesis_block.hash = genesis_block.compute_hash()
self.chain.append(genesis_block)

    @property
    def last_block(self):
        return self.chain[-1]

    def add_block(self, block, proof):
        """
        A function that adds the block to the chain after verification.
        Verification includes:
        * Checking if the proof is valid.
        * The previous_hash referred in the block and the hash of latest block
          in the chain match.
        """
previous_hash = self.last_block.hash

        if previous_hash != block.previous_hash:
            return False

        if not self.is_valid_proof(block, proof):
            return False

        block.hash = proof
        print("main "+str(block.hash))
        self.chain.append(block)
        return True

    def is_valid_proof(self, block, block_hash):
```

```

        """
        Check if block_hash is valid hash of block and satisfies
        the difficulty criteria.
        """
print("compare "+str(block_hash == block.compute_hash())+" "+block.compute_hash()+"
"+str(block_hash.startswith('0' * Blockchain.difficulty)))
        return (block_hash.startswith('0' * Blockchain.difficulty) and
block_hash == block.compute_hash())

    def proof_of_work(self, block):
        """
        Function that tries different values of nonce to get a hash
        that satisfies our difficulty criteria.
        """
block.nonce = 0

    computed_hash = block.compute_hash()
        while not computed_hash.startswith('0' * Blockchain.difficulty):
block.nonce += 1
    computed_hash = block.compute_hash()

        return computed_hash

    def add_new_transaction(self, transaction):
self.unconfirmed_transactions.append(transaction)

    def mine(self):
        """
        This function serves as an interface to add the pending
        transactions to the blockchain by adding them to the block
        and figuring out Proof of Work.
        """
        if not self.unconfirmed_transactions:
            return False

last_block = self.last_block

new_block = Block(index=last_block.index + 1,
                  transactions=self.unconfirmed_transactions,
                  timestamp=time.time(),
                  previous_hash=last_block.hash)

        proof = self.proof_of_work(new_block)
self.add_block(new_block, proof)

self.unconfirmed_transactions = []
        return proof

class Block:
    def __init__(self, index, transactions, timestamp, previous_hash):
self.index = index
self.transactions = transactions
self.timestamp = timestamp
self.previous_hash = previous_hash
self.nonce = 0

```

```

def compute_hash(self):
    """
    A function that return the hash of the block contents.
    """
    block_string = json.dumps(self.__dict__, sort_keys=True)
    return sha256(block_string.encode()).hexdigest()

def AccessData(request):
    if request.method == 'GET':
        return render(request, 'AccessData.html', {})

def index(request):
    if request.method == 'GET':
        return render(request, 'index.html', {})

def CreateProfile(request):
    if request.method == 'GET':
        return render(request, 'CreateProfile.html', {})

def Hospital(request):
    if request.method == 'GET':
        return render(request, 'Hospital.html', {})

def Patient(request):
    if request.method == 'GET':
        return render(request, 'Patient.html', {})

def PatientLogin(request):
    if request.method == 'POST':
        pid = request.POST.get('t1', False)
        strdata = '<table border=1 align=center width=100%><tr><th>Patient  
ID</th><th>Patient Name</th><th>Age</th><th>Problem Description</th><th>Profile  
Date</th><th>Access Control</th><th>Gender</th><th>Contact  
No</th><th>Address</th><th>Block Chain Hashcode</th><th>Revenue</th></th></tr><tr>'
        con = pymysql.connect(host='127.0.0.1',port = 3308,user = 'root', password =
        'root', database = 'SecuringData',charset='utf8')
        with con:
            cur = con.cursor()
        cur.execute("select * FROM patients")
        rows = cur.fetchall()
        for row in rows:
            if str(row[0]) == pid:
                strdata+='<td>'+str(row[0])+'</td><td>'+row[1]+'</td><td>'+str(row[2])+'</td><td>'+
                str(row[3])+'</td><td>'+str(row[4])+'</td><td>'+row[5]+'</td><td>'+row[6]+'</td><td>'+
                row[7]+'</td><td>'+row[8]+'</td><td>'+row[9]+'</td><td>'+str(row[10])+'</td></tr>
                <tr>'
                context= {'data':strdata}
                return render(request, 'ViewData.html', context)

def HospitalLogin(request):
    if request.method == 'POST':
        username = request.POST.get('t1', False)
        password = request.POST.get('t2', False)
        file = open('session.txt','w')
        file.write(username)
        file.close()

```

```

        if username == 'Hospital1' and password == 'Hospital1' or username ==
'Hospital2' and password == 'Hospital2':
            context= {'data': 'welcome '+username}
            return render(request, 'HospitalScreen.html', context)
        else:
            context= {'data': 'login failed'}
            return render(request, 'Hospital.html', context)

def updateRevenue(value):
    db_connection = pymysql.connect(host='127.0.0.1',port = 3306,user = 'root',
password = 'root', database = 'SecuringData',charset='utf8')
    db_cursor = db_connection.cursor()
    db_cursor.execute("update patients set revenue=revenue+0.5 where
patient_id='"+value+"'")
    db_connection.commit()

def PatientDataAccess(request):
    if request.method == 'POST':
        search = request.POST.get('t1', False)
        user = ''
        with open("session.txt", "r") as file:
            for line in file:
                user = line.strip('\n')
    strdata = '<table border=1 align=center width=100%><tr><th>Patient
ID</th><th>Patient Name</th><th>Age</th><th>Problem Description</th><th>Profile
Date</th><th>Access Control</th><th>Gender</th><th>Contact No</th></th></tr><tr>'
    con = pymysql.connect(host='127.0.0.1',port = 3308,user = 'root', password =
'root', database = 'SecuringData',charset='utf8')
    print("select * FROM patients where problem_desc like '%" +search+"%'")
    with con:
        cur = con.cursor()
    cur.execute("select * FROM patients where problem_desc like '%" +search+"%'")
    rows = cur.fetchall()
    for row in rows:
        arr = row[5].split(" ")
        if len(arr) == 1:
            if arr[0] == user:
                updateRevenue(str(row[0]))

    strdata+='<td>'+str(row[0])+'</td><td>'+row[1]+'</td><td>'+str(row[2])+'</td><td>'+
str(row[3])+'</td><td>'+str(row[4])+'</td><td>'+row[5]+'</td><td>'+row[6]+'</td><td>'+
row[7]+'</td></tr><tr>'
        if len(arr) == 2:
            if arr[0] == user or arr[1] == user:
                updateRevenue(str(row[0]))

    strdata+='<td>'+str(row[0])+'</td><td>'+row[1]+'</td><td>'+str(row[2])+'</td><td>'+
str(row[3])+'</td><td>'+str(row[4])+'</td><td>'+row[5]+'</td><td>'+row[6]+'</td><td>'+
row[7]+'</td></tr><tr>'
        context= {'data': strdata}
        return render(request, 'ViewAccessData.html', context)

def CreateProfileData(request):
    if request.method == 'POST':
        name = request.POST.get('t1', False)
        age = request.POST.get('t2', False)

```

```

        problem = request.POST.get('t3', False)
access_list = request.POST.getlist('t4', False)
        gender = request.POST.get('t5', False)
        contact = request.POST.get('t6', False)
        address = request.POST.get('t7', False)
        revenue = 0
        count = 0
        access = ''
        for i in range(len(access_list)):
            access+=access_list[i]+" "
        access = access.strip()
db_connection = pymysql.connect(host='127.0.0.1',port = 3308,user = 'root',
password = 'root', database = 'SecuringData',charset='utf8')
        with db_connection:
            cur = db_connection.cursor()
cur.execute("select count(*) FROM patients")
            rows = cur.fetchall()
            for row in rows:
                count = row[0]
            count = count + 1
            now = datetime.datetime.now()
current_time = now.strftime("%Y-%m-%d %H:%M:%S")

        blockchain = Blockchain()
        x = '{ "Patient_id":"' +str(count)+'"', "patient_name":"' +name+'"',
"age":"' +age+'"', "problem_desc":"' +problem+'"',
"profile_date":"' +str(current_time)+'"',
"access_data":"' +str(access)+'"', "gender":"' +gender+'"}'
blockchain.add_new_transaction(json.loads(x))
        hash = blockchain.mine()

db_cursor = db_connection.cursor()
student_sql_query = "INSERT INTO
patients(Patient_id,patient_name,age,problem_desc,profile_date,access_data,gender,c
ontact_no,address,blockchain_hash,revenue)
VALUES('"+str(count)+"','"+name+"','"+age+"','"+problem+"','"+current_time+"','"+st
r(access)+"','"+gender+"','"+contact+"','"+address+"','"+hash+"','"+str(revenue)+"'
)"
db_cursor.execute(student_sql_query)
db_connection.commit()
print(db_cursor.rowcount, "Record Inserted")
        if db_cursor.rowcount == 1:
            context= {'data':'Profile Creation Process Completed. Your Patient ID :
'+str(count)}
            return render(request, 'CreateProfile.html', context)
        else:
            context= {'data':'Error in profile creation process'}
            return render(request, 'CreateProfile.html', context)

```

apps.py

```
from django.apps import AppConfig
```

```
class SecureConfig(AppConfig):  
    name = 'Secure'
```

settings.py

```
import os

# Build paths inside the project like this: os.path.join(BASE_DIR, ...)
BASE_DIR = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))

# Quick-start development settings - unsuitable for production
# See https://docs.djangoproject.com/en/2.2/howto/deployment/checklist/

# SECURITY WARNING: keep the secret key used in production secret!
SECRET_KEY = 'mgdbhpmqsa%b$kfz7cntiy*vkyo%y#j$)r#ldo)bx)w9tj1l@n'

# SECURITY WARNING: don't run with debug turned on in production!
DEBUG = True ALLOWED_HOSTS = []

# Application definition

INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'Secure'
]

MIDDLEWARE = [
    'django.middleware.security.SecurityMiddleware',
    'django.contrib.sessions.middleware.SessionMiddleware',
    'django.middleware.common.CommonMiddleware',
    'django.middleware.csrf.CsrfViewMiddleware',
    'django.contrib.auth.middleware.AuthenticationMiddleware',
    'django.contrib.messages.middleware.MessageMiddleware',
    'django.middleware.clickjacking.XFrameOptionsMiddleware',
]

ROOT_URLCONF = 'DataSecuring.urls'
TEMPLATES = [
    {
        'BACKEND':
            'django.template.backends.django.DjangoTemplates',
        'DIRS':
            [
os.path.join('C:/Python/Secure', 'templates'),
            ],
        'APP_DIRS': True,
        'OPTIONS': {
            'context_processors': [
                'django.template.context_processors.debug',
                'django.template.context_processors.request',
                'django.contrib.auth.context_processors.auth',
                'django.contrib.messages.context_processors.messages',
            ],
        },
    },
]
```

```

        ],
    },
],

WSGI_APPLICATION = 'DataSecuring.wsgi.application'

# Database
# https://docs.djangoproject.com/en/2.2/ref/settings/#databases

DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.mysql',
        'NAME': 'SecuringData',
        'HOST': '127.0.0.1',
        'PORT': '3307',
        'USER': 'root',
        'PASSWORD': 'root',
        'OPTIONS': {
            'autocommit': True,
        },
    }
}

# Password validation
# https://docs.djangoproject.com/en/2.2/ref/settings/#auth-password-validators

AUTH_PASSWORD_VALIDATORS = [
    {
        'NAME':
            'django.contrib.auth.password_validation.UserAttributeSimilarityValidator',
    },
    {
        'NAME': 'django.contrib.auth.password_validation.MinimumLengthValidator',
    },
    {
        'NAME': 'django.contrib.auth.password_validation.CommonPasswordValidator',
    },
    {
        'NAME': 'django.contrib.auth.password_validation.NumericPasswordValidator',
    },
]

```


urls.py

```
from django.urls import path

from . import views

urlpatterns = [path("index.html", views.index, name="index"),
path("CreateProfile.html", views.CreateProfile,
name="CreateProfile"),
    path("CreateProfileData", views.CreateProfileData, name="CreateProfileData"),
    path("Hospital.html", views.Hospital, name="Hospital"),
    path("HospitalLogin", views.HospitalLogin, name="HospitalLogin"),
    path("AccessData.html", views.AccessData, name="AccessData"),
    path("PatientDataAccess", views.PatientDataAccess,
name="PatientDataAccess"), path("Patient.html", views.Patient,
name="Patient"),
    path("PatientLogin", views.PatientLogin, name="PatientLogin"),
]
```

Chapter 5

Testing

5.1 Testing Approach

The purpose of testing is to discover errors. Testing is the process of trying to discover every conceivable fault or weakness in a work product. It provides a way to check the functionality of components, sub-assemblies, assemblies and/or a finished product. It is the process of exercising software with the intent of ensuring that the Software system meets its requirements and user expectations and does not fail in an unacceptable manner. There are various types of tests. Each test type addresses a specific testing requirement.

TYPES OF TESTS

5.1.1 Unit testing

Unit testing involves the design of test cases that validate that the internal program logic is functioning properly, and that program inputs produce valid outputs. All decision branches and internal code flow should be validated. It is the testing of individual software units of the application. It is done after the completion of an individual unit before integration. This is a structural testing, that relies on knowledge of its construction and is invasive. Unit tests perform basic tests at component level and test a specific business process, application, and/or system configuration. Unit tests ensure that each unique path of a business process performs accurately to the documented specifications and contains clearly defined inputs and expected results.

5.1.2 Integration testing

Integration tests are designed to test integrated software components to determine if they actually run as one program. Testing is event driven and is more concerned with the basic outcome of screens or fields. Integration tests demonstrate that although the components were individually satisfactory, as shown by successfully unit testing, the combination of components is correct and

consistent. Integration testing is specifically aimed at exposing the problems that arise from the combination of components.

5.1.3 Functional test

Functional tests provide systematic demonstrations that functions tested are available as specified by the business and technical requirements, system documentation, and user manuals.

Functional testing is centered on the following items:

- Valid Input : identified classes of valid input must be accepted. Invalid
- Input : identified classes of invalid input must be rejected.
- Functions : identified functions must be exercised.
- Output : identified classes of application outputs must be exercised.
- Systems/Procedures : interfacing systems or procedures must be invoked.

Organization and preparation of functional tests is focused on requirements, key functions, or special test cases. In addition, systematic coverage pertaining to identify Business process flows; data fields, predefined processes, and successive processes must be considered for testing. Before functional testing is complete, additional tests are identified and the effective value of current tests is determined.

*** System Test**

System testing ensures that the entire integrated software system meets requirements. It tests a configuration to ensure known and predictable results. An example of system testing is the configuration-oriented system integration test. System testing is based on process descriptions and flows, emphasizing pre-driven process links and integration points.

*** White Box Testing**

White Box Testing is a testing in which in which the software tester has knowledge of the inner workings, structure and language of the software, or at least its purpose. It is purpose. It is used to test areas that cannot be reached from a black box level

* **Black Box Testing**

Black Box Testing is testing the software without any knowledge of the inner workings, structure or language of the module being tested. Black box tests, as most other kinds of tests, must be written from a definitive source document, such as specification or requirements document, such as specification or requirements document. It is a testing in which the software under test is treated, as a black box. You cannot “see” into it. The test provides inputs and responds to outputs without considering how the software works.

* **Unit Testing**

Unit testing is usually conducted as part of a combined code and unit test phase of the software lifecycle, although it is not uncommon for coding and unit testing to be conducted as two distinct phases.

Test strategy and approach

Field testing will be performed manually and functional tests will be written in detail.

Test objectives

- All field entries must work properly.
- Pages must be activated from the identified link.
- The entry screen, messages and responses must not be delayed.

Features to be tested

- Verify that the entries are of the correct format
- No duplicate entries should be allowed
- All links should take the user to the correct page.

Integration Testing

Software integration testing is the incremental integration testing of two or more integrated software components on a single platform to produce failures caused by interface defects.

The task of the integration test is to check that components or software applications, e.g., components in a software system or – one step up – software applications at the company level – interact without error.

Test Results: All the test cases mentioned above passed successfully. No defects encountered.

Acceptance Testing

User Acceptance Testing is a critical phase of any project and requires significant participation by the end user. It also ensures that the system meets the functional requirements.

Test Results: All the test cases mentioned above passed successfully. No defects encountered.

Chapter 6

Results and Discussions

Deploy code on DJANGO and start server and run-in browser to get below screen

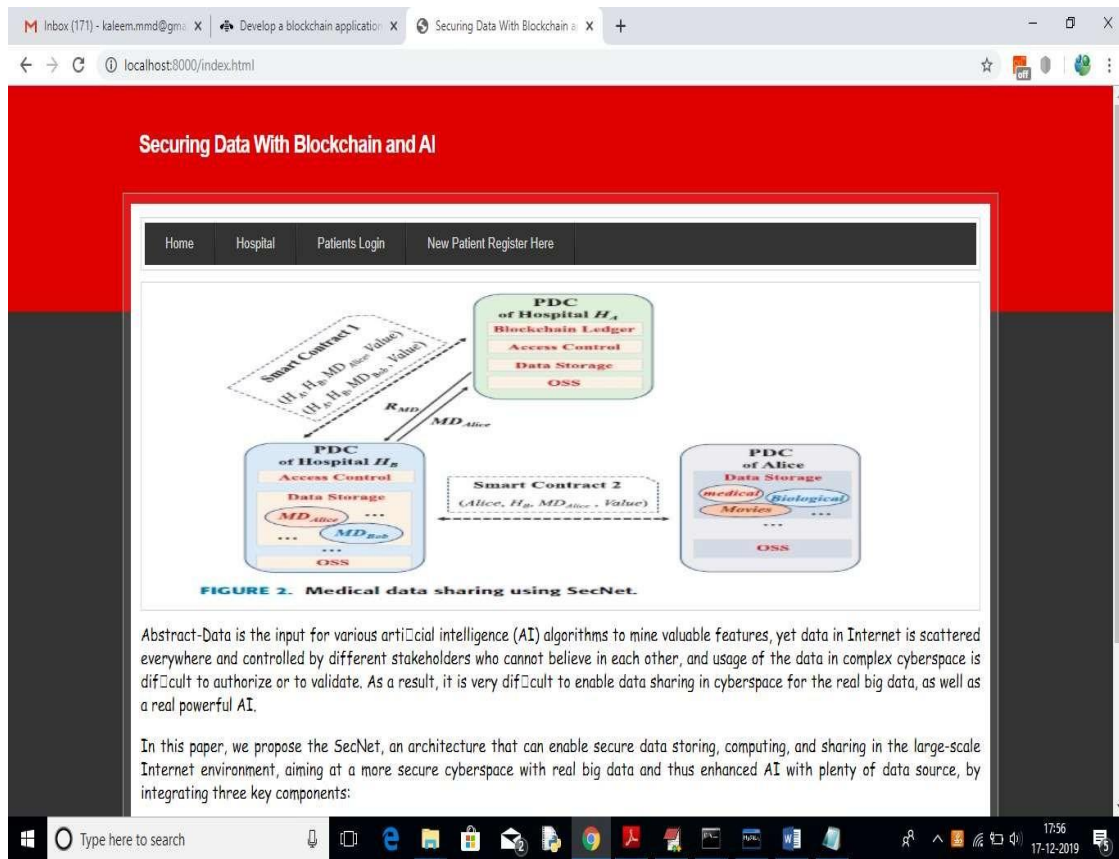


Fig 6.1 index page

In above screen click on 'New Patient Register Here' link to get below screen

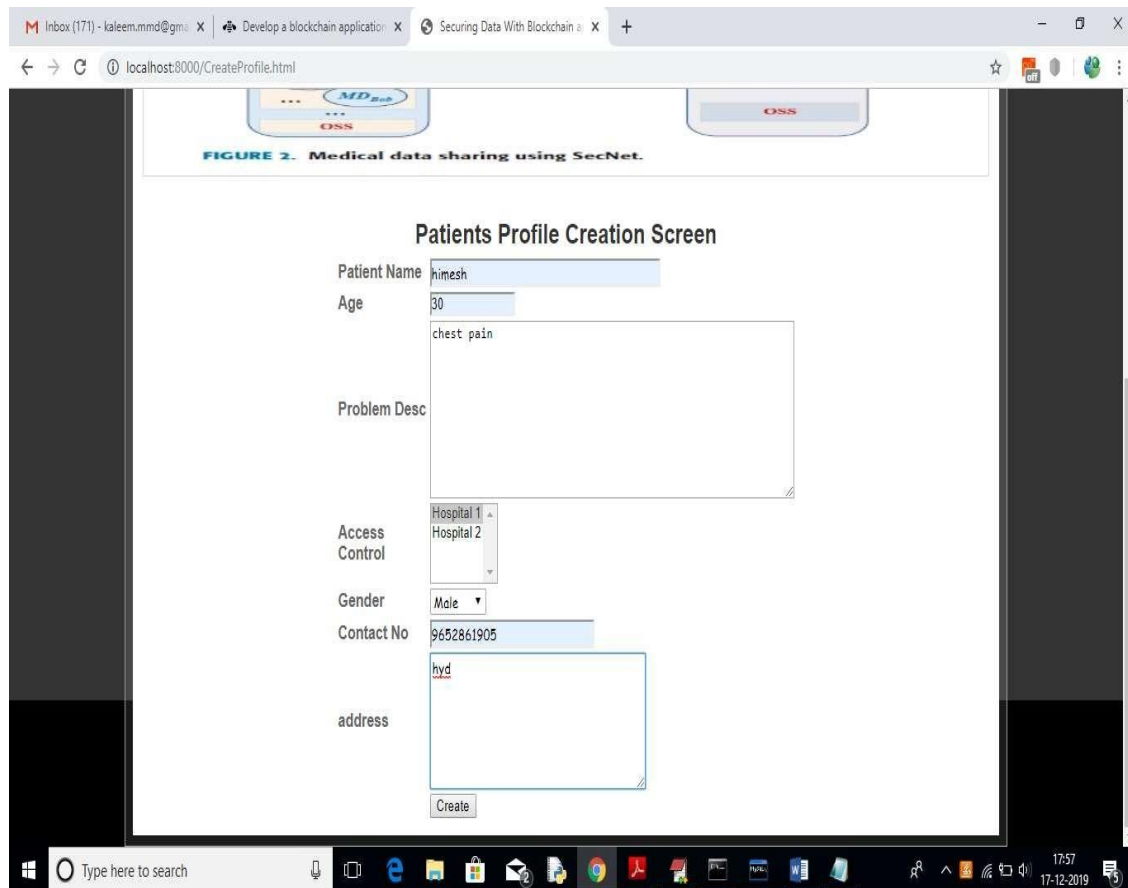


Fig 6.2 Patient New Registration

In above screen I am adding patient disease details and selecting 'Hospital1' to share my data and if you want to share with two hospitals then hold 'CTRL' key and select both hospitals to give permission. Now press 'Create' button to create profile.

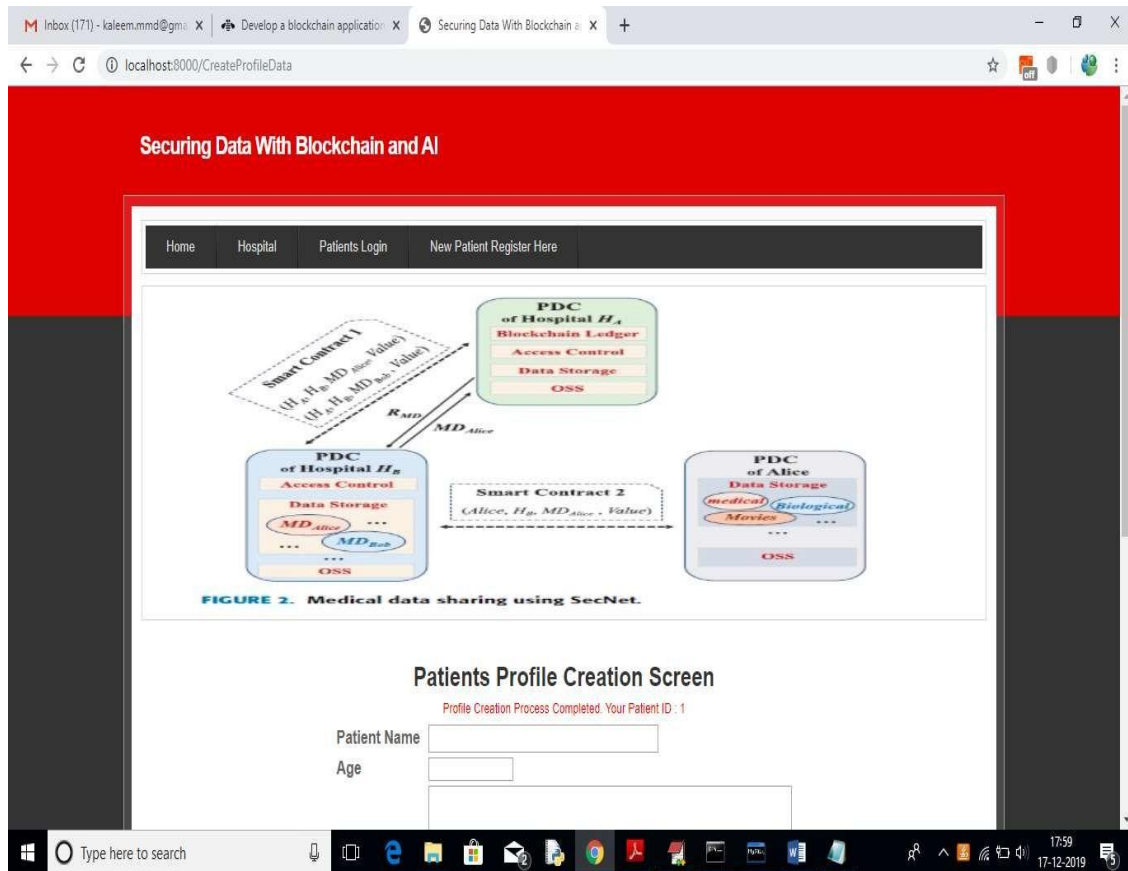


Fig 6.3 Patient profile creation

In above screen one patient is created with patient ID 1 and now Hospital 1 can login and search and access this patient data as patient has given permission to Hospital1

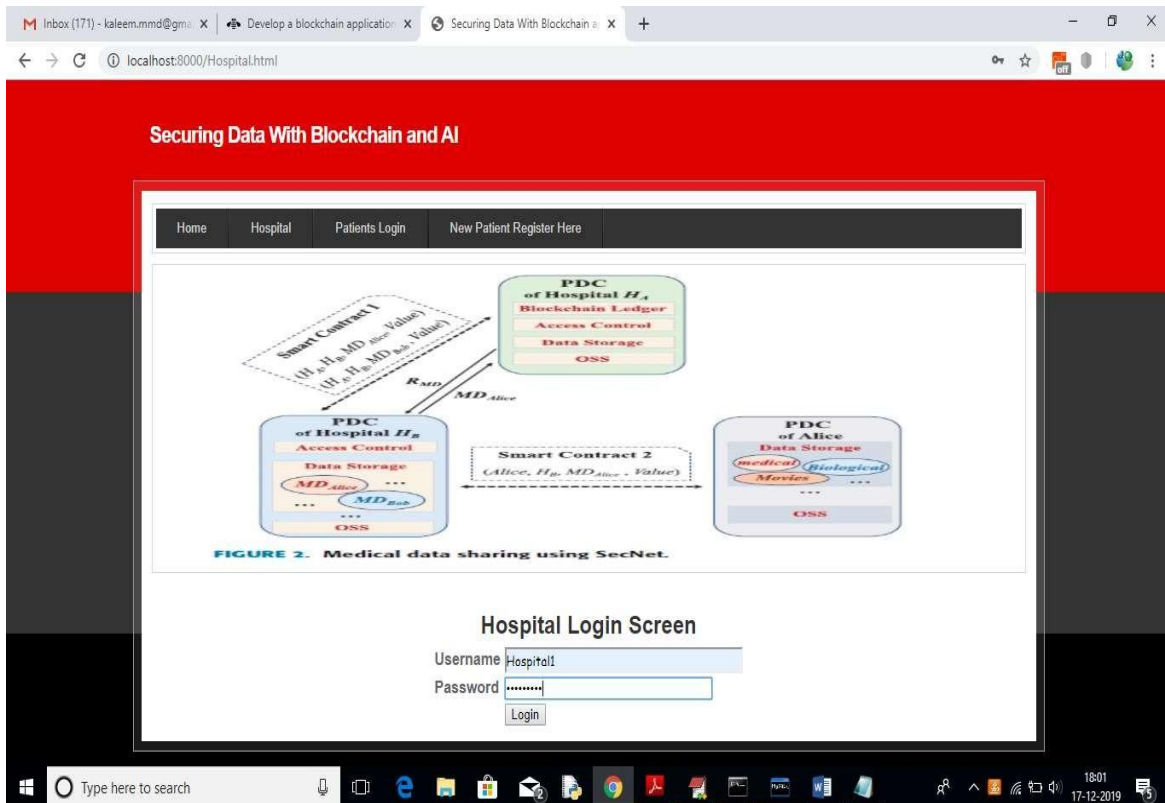


Fig 6.4 Hospital1 login

In above screen to login as Hospital1 click on 'Hospital' link to get above screen. Use 'Hospital1' as username and 'Hospital1' as password to login as Hospital1 and use Hospital2 to login as Hospital2. After login will get below screen

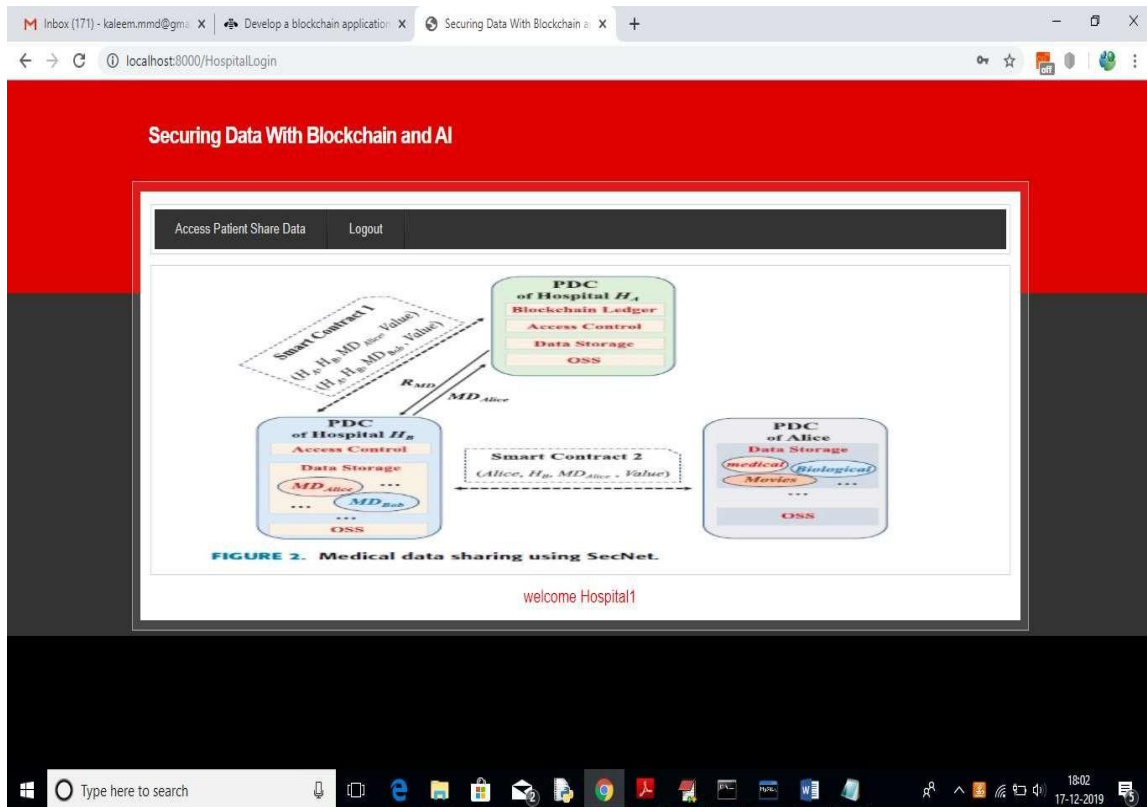


Fig 6.5 Hospital1 welcome page

In above screen click on 'Access Patient Share Data' link to search for patient details

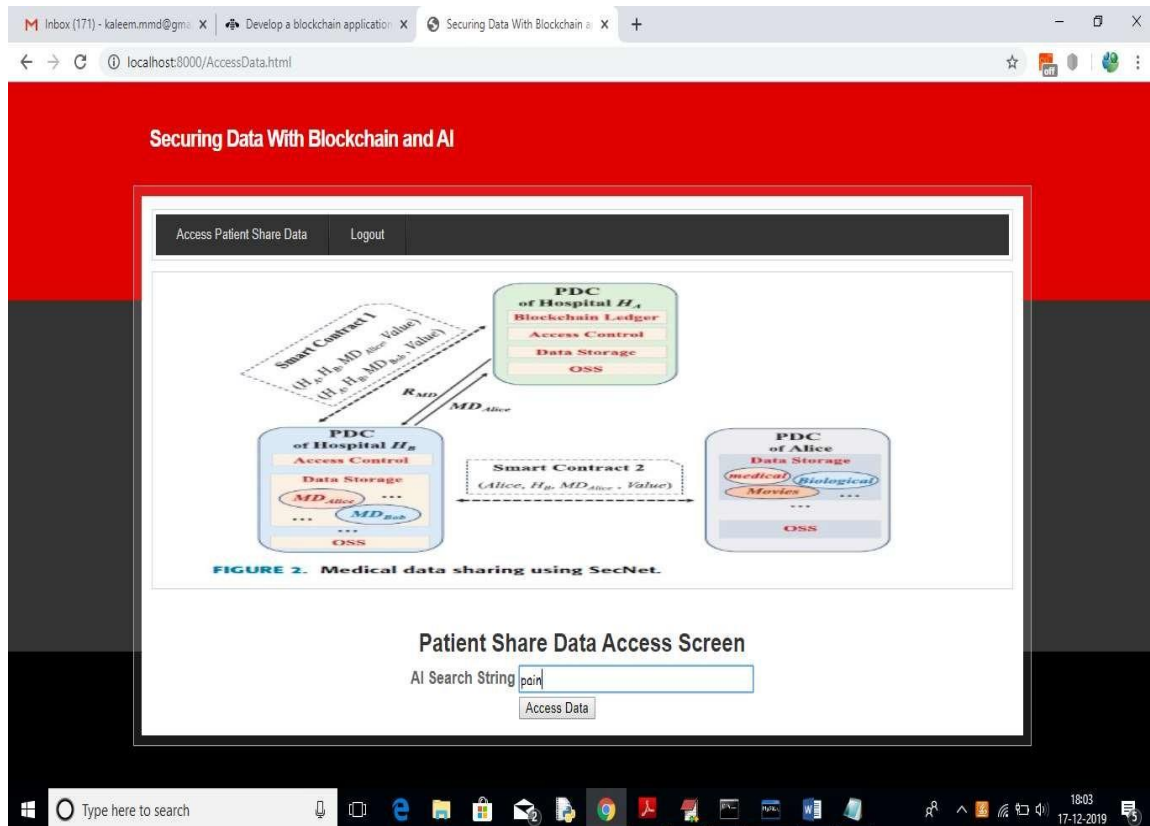


Fig 6.6 Access Patient share Data

In above screen I want to search for all patients who are suffering from 'pain' and then clickon 'Access data' button to get below screen

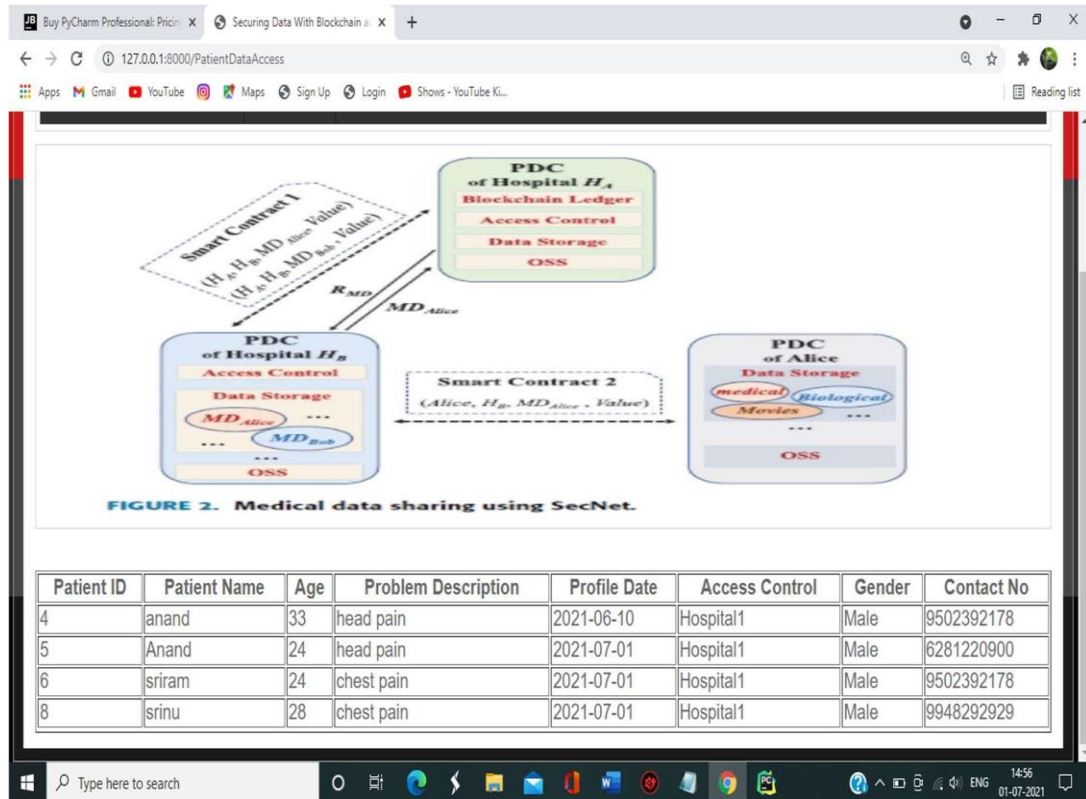


Fig 6.7 Patient Data Access

In above screen Hospital1 is getting details of patient and Hospital2 for not having permission so it will not get details. To see this logout and login as 'Hospital2'

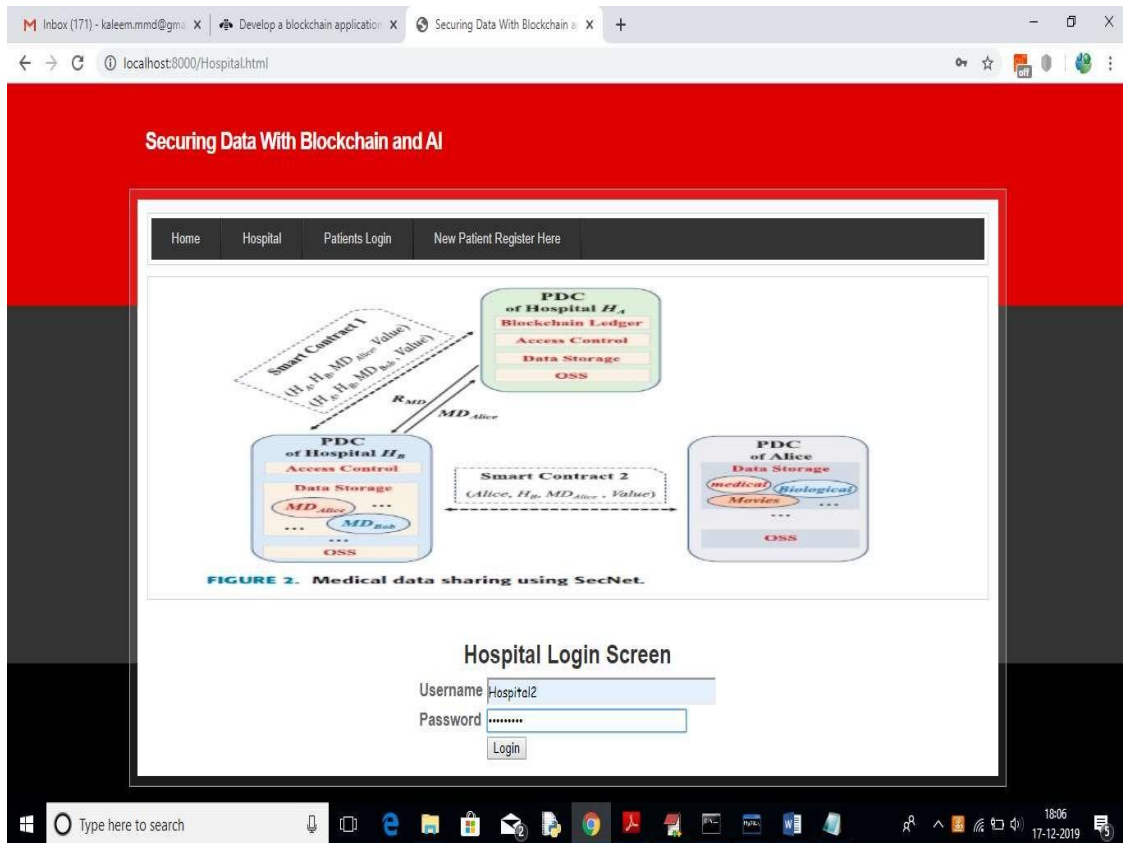


Fig 6.8 Hospital 2 Login

In above screen 'Hospital2' is login, after login will get below screen

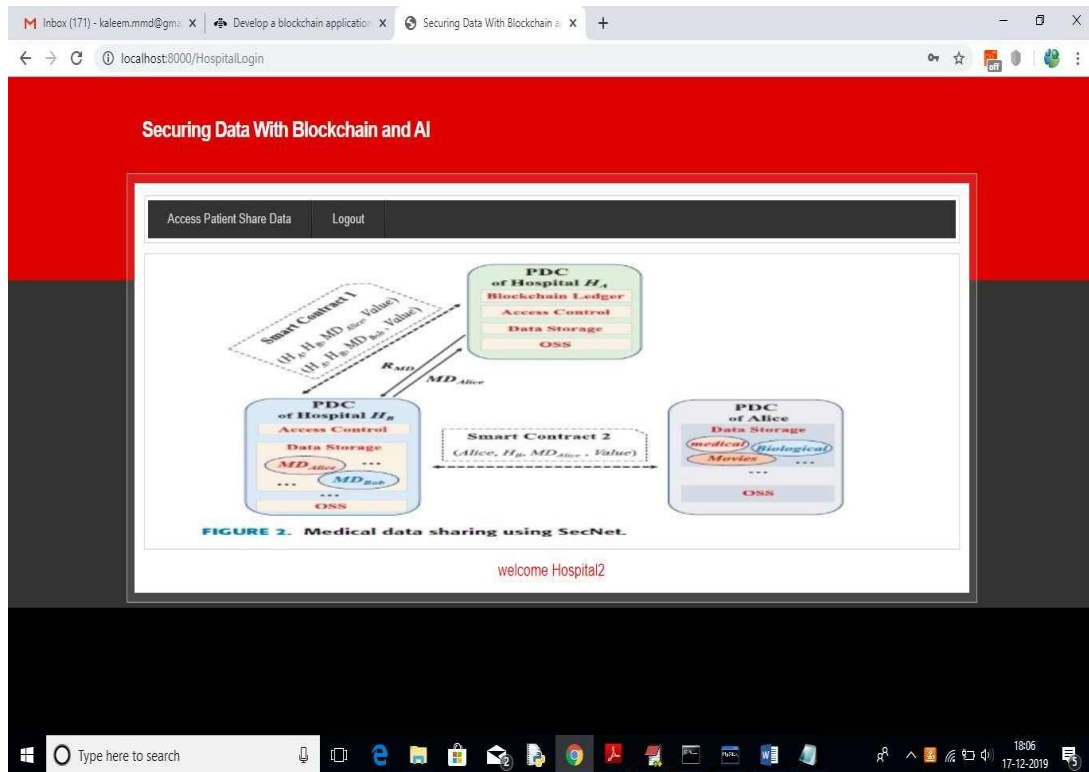


Fig 5.9 Hospital2 welcome

Now click on ‘Access Patient Share Data’ link and search for same pain disease

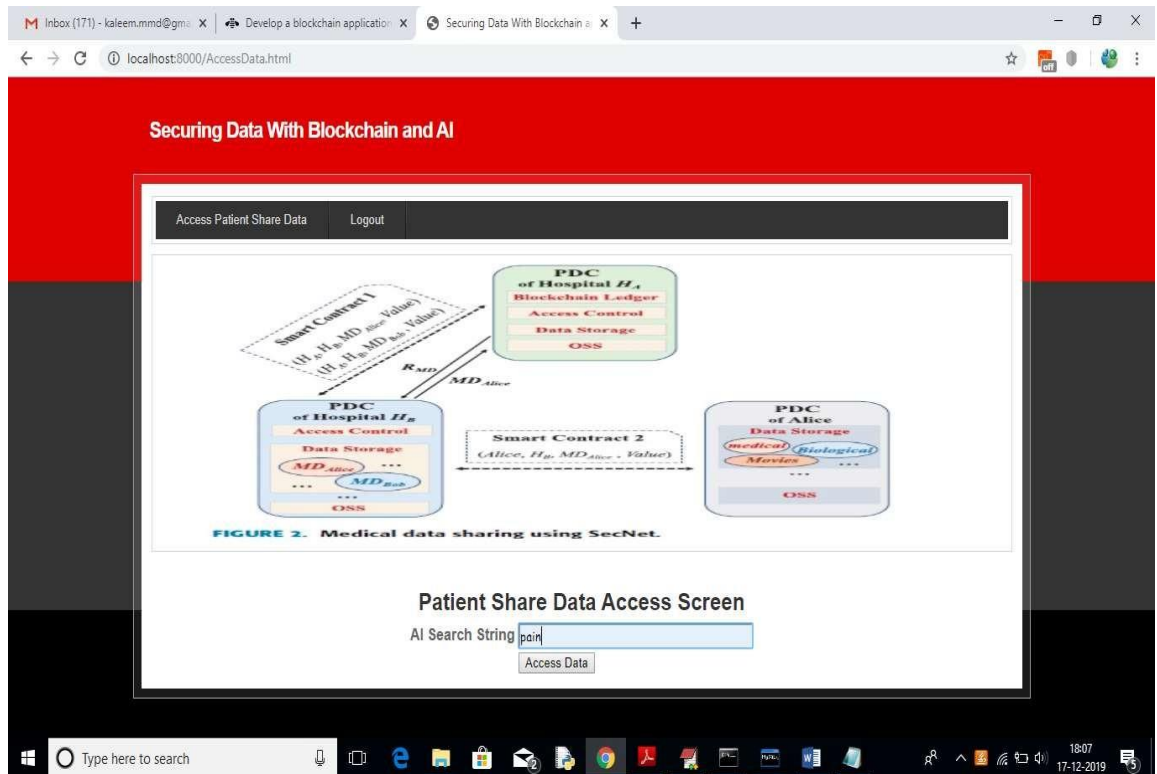


Fig 6.10 Access Patient data

For above query will get below result

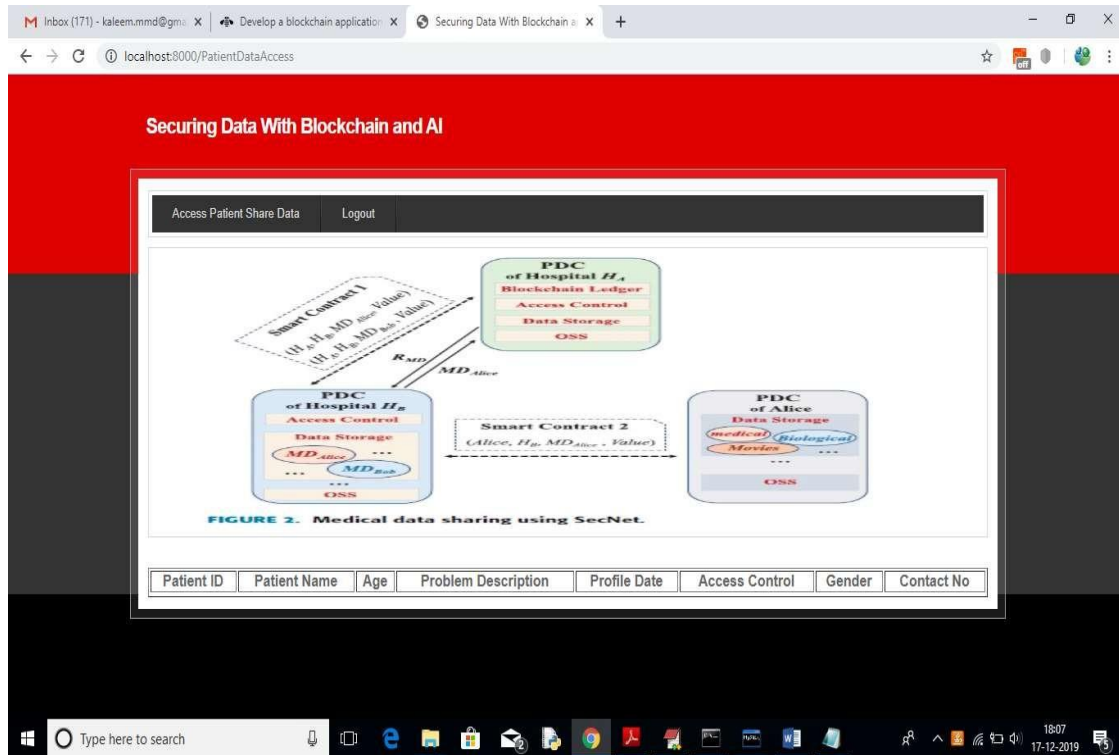


Fig 6.11 Access Data

In above screen no patient details are showing as Hospital2 not having permission. So, block chain allows only those users to access data who has permission. Now logout and login as patient by entering patient id in below screen

Securing Data With Blockchain and AI

Home Hospital Patients Login New Patient Register Here

Smart Contract 1
 $(H_1, H_2, MD_{Alice}, Value)$
 $(H_1, H_2, MD_{H_2}, Value)$

PDC of Hospital H_1
 Blockchain Ledger
 Access Control
 Data Storage
 OSS

PDC of Hospital H_2
 Access Control
 Data Storage
 MD_{Alice}
 MD_{Bob}
 OSS

Smart Contract 2
 $(Alice, H_2, MD_{Alice}, Value)$

PDC of Alice
 Data Storage
 $medical$
 $biological$
 $Movies$
 OSS

FIGURE 2. Medical data sharing using SecNet.

Patient Login Screen

Patient ID

Login

Fig 6.12 Patient login

After login will get below details for patient 1

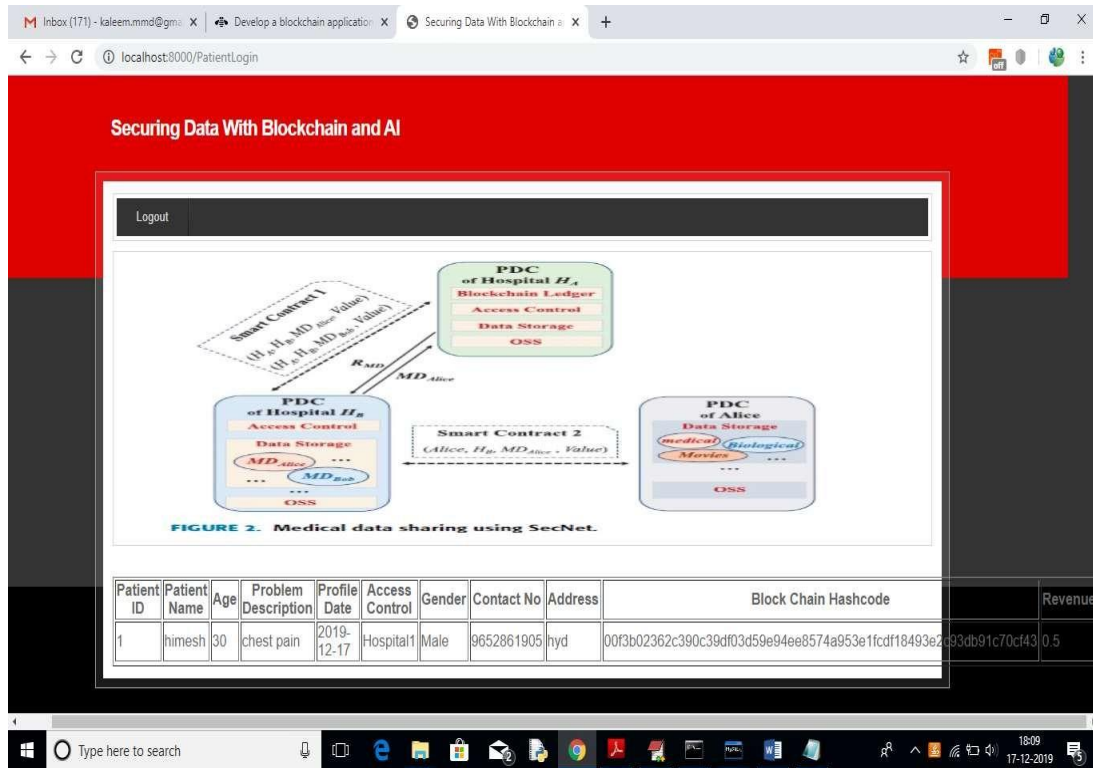


Fig 6.13 Patient ID1 Data

In above screen we can see patient all details and hash code generated by block chain and in last column we can see patient reward revenue as 0.5 and it will get update upon every access from hospital user.

Chapter 7

Conclusion

Conclusion

In order to leverage AI and blockchain to fit the problem of abusing data, as well as empower AI with the help of blockchain for trusted data management in trust-less environment we propose the SecNet, which is a new network paradigm focusing on secure data storing, sharing and computing instead of communicating. SecNet provides data ownership guaranteeing with the help of blockchain technologies, and AI-based secure computing platform as well as blockchain-based incentive mechanism, offering paradigm and incentives for data merging and more powerful AI to finally achieve better network security.

Moreover, we discuss the typical use scenario of SecNet in medical care system, and gives alternative ways for employing the storage function of SecNet. We can see patient all details and hash code generated by block chain and in last column we can see patient reward revenue as 0.5 and it will get update upon every access from hospital user.

Chapter 8

References

- [1] <https://www.python.org/downloads/>

- [2] <https://www.jetbrains.com/pycharm/>

- [3] <https://www.djangoproject.com/>

- [4] https://www.tutorialspoint.com/python_blockchain/index.htm

- [5] https://www.tutorialspoint.com/artificial_intelligence_with_python/artificial_intelligence_with_python_heuristic_search.htm

- [6] <https://www.w3schools.com/MySQL/default.asp>

- [7] <https://pynative.com/python-mysql-database-connection/>

- [8] <https://www.javatpoint.com/blockchain-hash-function>